02285 AI and MAS, F26
Planning based on Iterated Width

Curriculum for week 5: `lipovetzky2012width.pdf`, the paper *Width and Serialization of Classical Planning Problems* by Nir Lipovetzky and Hector Geffner (available on DTU Learn). *Optional*: Chapters 6–8 of `lipovetzky2014structure.pdf`, the PhD thesis of Nir Lipovetzky.

Taming the combinatorial complexity of planning

In STRIPS/PDDL planning, the size of the state space is (at most) exponential in the number of ground atoms. Why? A state is a subset of ground atoms, and the number of subsets of a set X is $2^{|X|}$.

Example. Consider a blocks world problem with n distinct blocks and the usual predicates $On(x, y)$, $Block(x)$ and $Clear(x)$. How many ground atoms do we have (ignoring the rigids)? *Block* is rigid, so doesn't count. We have $n + 1$ atoms for *Clear*, one for each block, and one extra for the *Table*. We have $(n + 1)^2$ atoms for *On*. So in total $(n + 1) + (n + 1)(n + 1) = (n + 1)(n + 2)$. For $n = 3$ that we will consider later, we hence have $|At| = 4 * 5 = 20$.

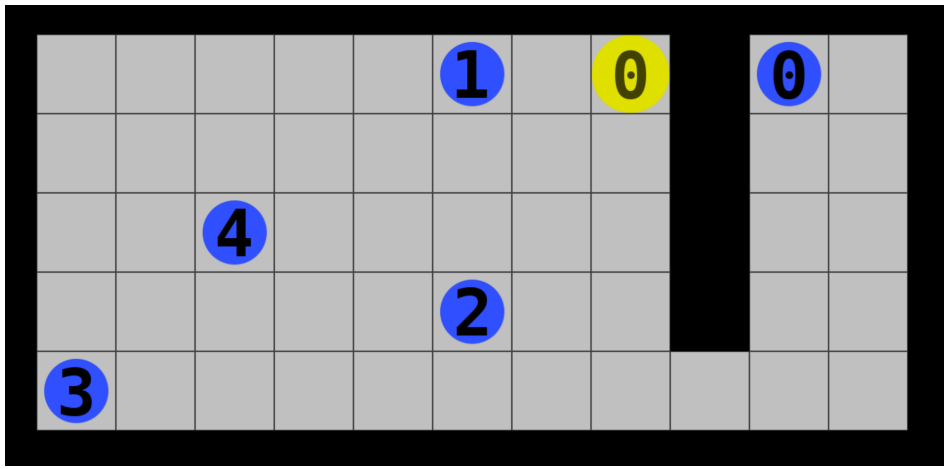
Issue: A given blocks world problem with n blocks might be simple, but the state space still grows (super-)exponentially with the number of blocks.

Standard reply: Use heuristics to guide the search, to only explore the "relevant" parts of the state space. However...

Potential issue with relying only on heuristics

Problem with standard reply: Sometimes it's hard to come up with good heuristics and/or the automatically induced heuristics from the PDDL description are not very efficient.

Even good heuristics might get trapped in **heuristic plateaus**, like this level when using greedy best-first search with a Manhattan distance heuristic:



Possible alternative reply: What if we simply reduce the state space by only worrying about *some* of the atoms? In the example above, the state space would be much smaller if we only worried about the position of agent 0.

Substates and T^i

Definition (T^i, substates and containment)

Let P be a planning problem with set of ground atoms At. Define T^i as the set of subsets of At of size $\leq i$, called **substates** of P (of size $\leq i$). A state s of P **contains** a substate t if $t \subseteq s$.

Example: If $At = \{p_1, p_2, p_3\}$ then $T^2 = \{\emptyset, \{p_1\}, \{p_2\}, \{p_3\}, \{p_1, p_2\}, \{p_1, p_3\}, \{p_2, p_3\}\}$, and the state $\{p_1, p_2\}$ contains the substate $\{p_1\}$.

Example. In the MAPF problem above, each element of T^1 would only be able to provide the location of one of the agents.

We will try to solve planning problems by building graphs over T^i. *Advantage*: The size of T^i is exponential in i rather than in $|At|$, since $|T^i| \leq (|At| + 1)^i$. How is it seen that $|T^i| \leq (|At| + 1)^i$ holds? For substates of size $\leq i$, we i times have to choose either an element in At or nothing.

The graph $\mathcal{G}^i$

As for states, we can identify substates with the conjunction of the atoms they contain. Thus any substate is also a (conjunctive) formula.

Definition (Notation $P(t)$)

For a planning problem P and a substate t, $P(t)$ denotes the planning problem where the goal has been replaced by t.

Definition (The graph $\mathcal{G}^i$)

$\mathcal{G}^i$ is the smallest graph over T^i satisfying:

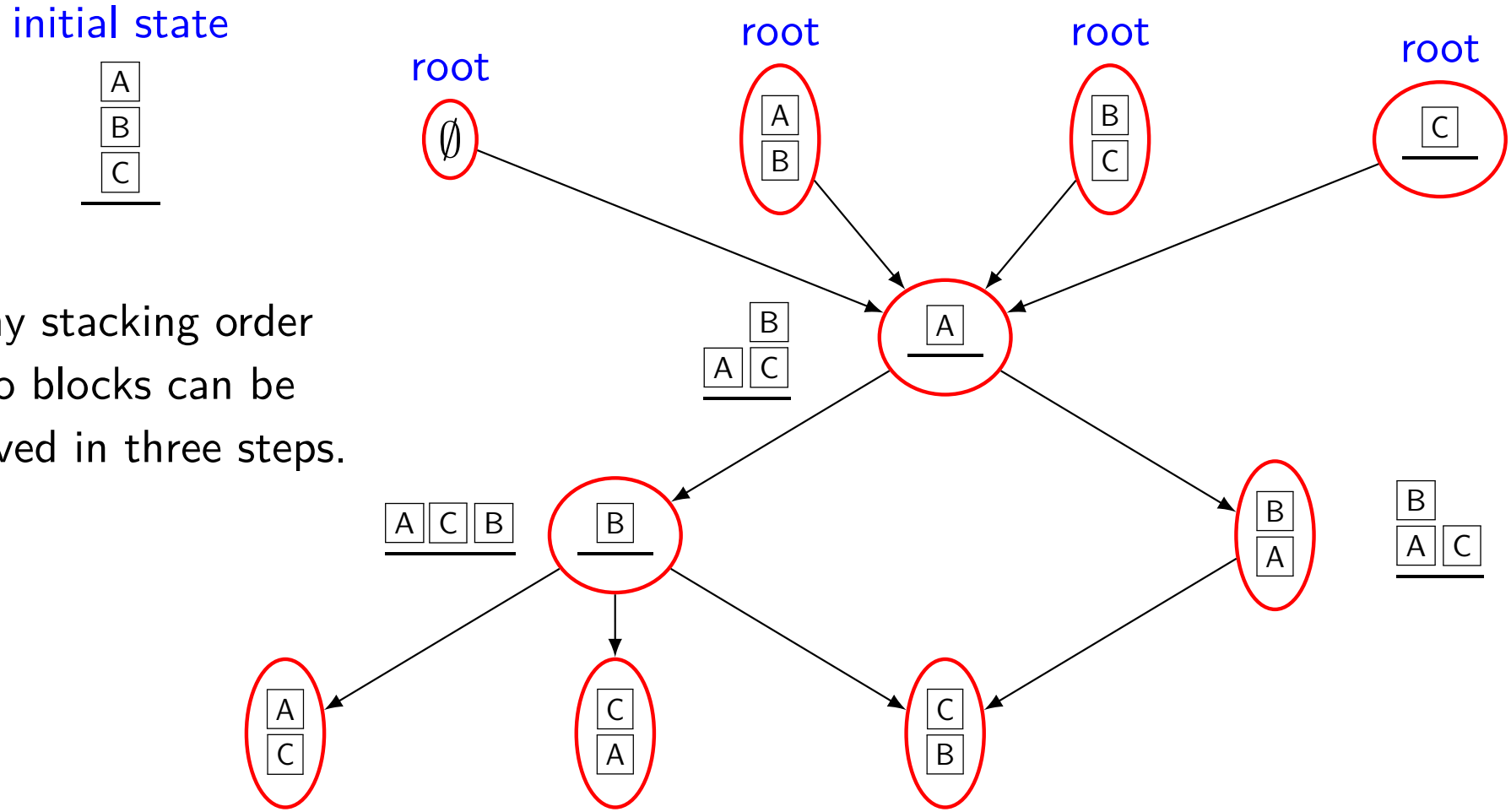
1. t is a root vertex iff t is contained in s_0.
2. There is an edge from t to t' iff every optimal plan for $P(t)$ can be extended with a single action to give an optimal plan for $P(t')$.

Consequence: If there is a path from a root vertex to a vertex satisfying the goal, then we have an optimal solution (of that length)!

$\mathcal{G}^1$ example

Recall: $\mathcal{G}^i$ is the smallest graph over T^i satisfying:

1. t is a root vertex iff t is contained in s_0.
2. There is an edge from t to t' iff every optimal plan for $P(t)$ can be extended with a single action to give an optimal plan for $P(t')$.



So any stacking order of two blocks can be achieved in three steps.

Width of a problem

Definition (Optimally implies)

Given a planning problem P, a formula t is said to **optimally imply** a formula t' if all optimal plans for $P(t)$ are also optimal for $P(t')$.

Example. In the blocks world domain, if $On(B, A)$ is initially true, then $On(B, Table)$ optimally implies $Clear(A)$.

Definition (Width of a planning problem)

The **width** of a planning problem $P = (\mathcal{A}, s_0, g)$ is the minimal value $w \in \mathbb{N}$ such that $\mathcal{G}^w$ contains a vertex t that optimally implies g.

Theorem

Any planning problem $P = (\mathcal{A}, s_0, g)$ can be solved optimally in exponential time in its width w (by computing $\mathcal{G}^w$ and finding a vertex that optimally implies g).

Good news for problems of low width. All blocks world problems, gripper problems and sliding puzzle problems have width 2 (for singleton goals).

Iterated Width

Computing $\mathcal{G}^w$ is non-trivial. Simpler approach also giving optimal solutions: Do normal BFS, but **block** a state from being added to the search tree if all of its T^w substates are contained in existing tree states. Is it OK to block these states? Yes, since for each T^w state they contain, the tree already has an optimal path to it. Does the resulting tree have size $\leq |T^w|$? Yes, every non-blocked state contains at least one new T^w substate.

Definition (Iterated width algorithm: $IW(i)$ and IW)

$IW(i)$ is the algorithm that runs like BFS, except a child state s is only added to the frontier if it contains at least one new T^i substate that is not already contained in any of the existing tree nodes. **Iterated Width** (or just **IW**) is the algorithm that runs $IW(0), IW(1), IW(2), \ldots$ until a goal state is discovered.

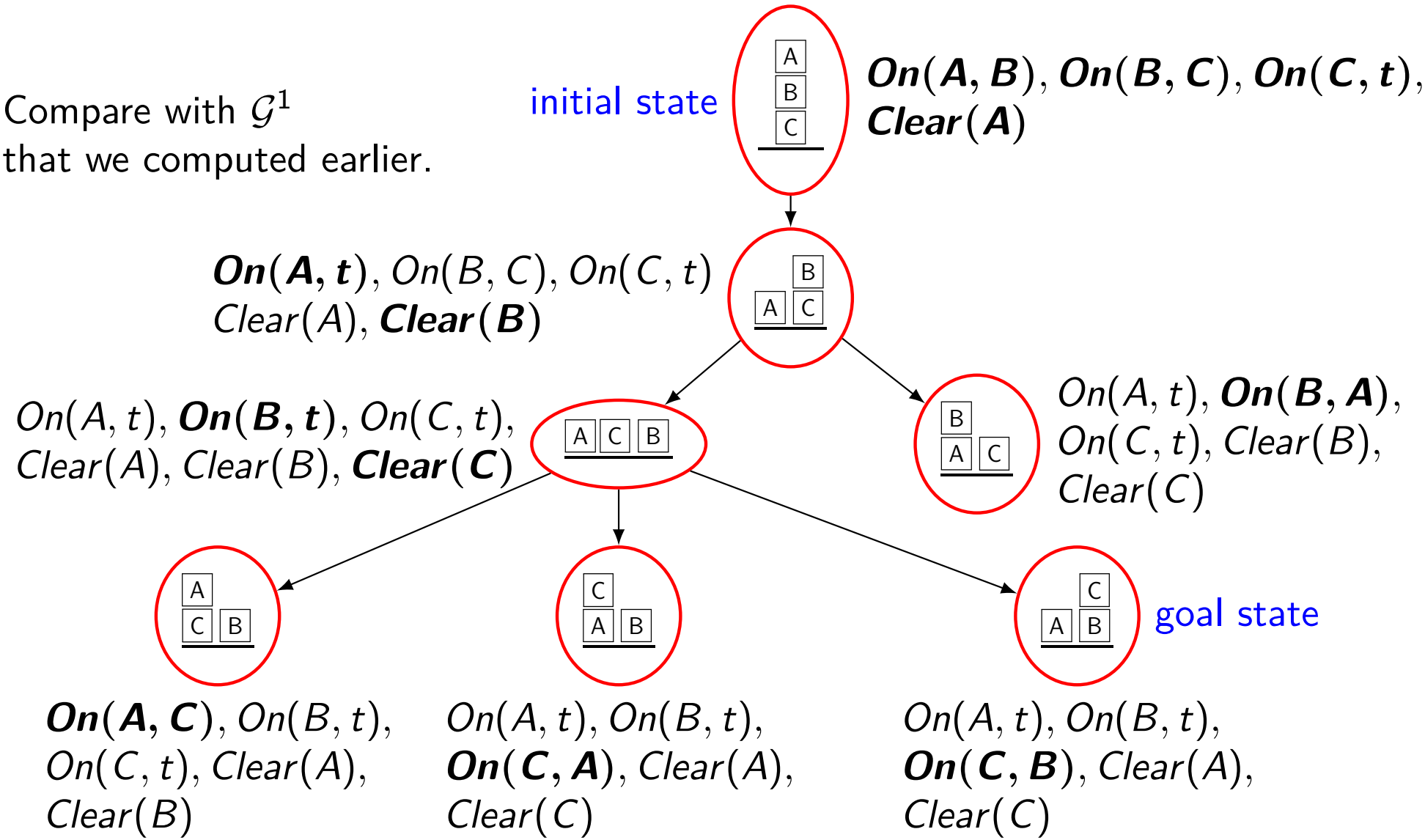
Theorem

*Let P be a planning problem of width w. Then $IW(w)$ solves it optimally. (Hence IW is **complete** and runs in exponential time in w.)*

Example (by Frederik Drachmann)

Blocks world problem with goal $On(C, B)$. $IW(0)$: no solution. $IW(1)$: Only add child state to frontier if it contains a previously unseen atom.

Compare with $\mathcal{G}^1$ that we computed earlier.



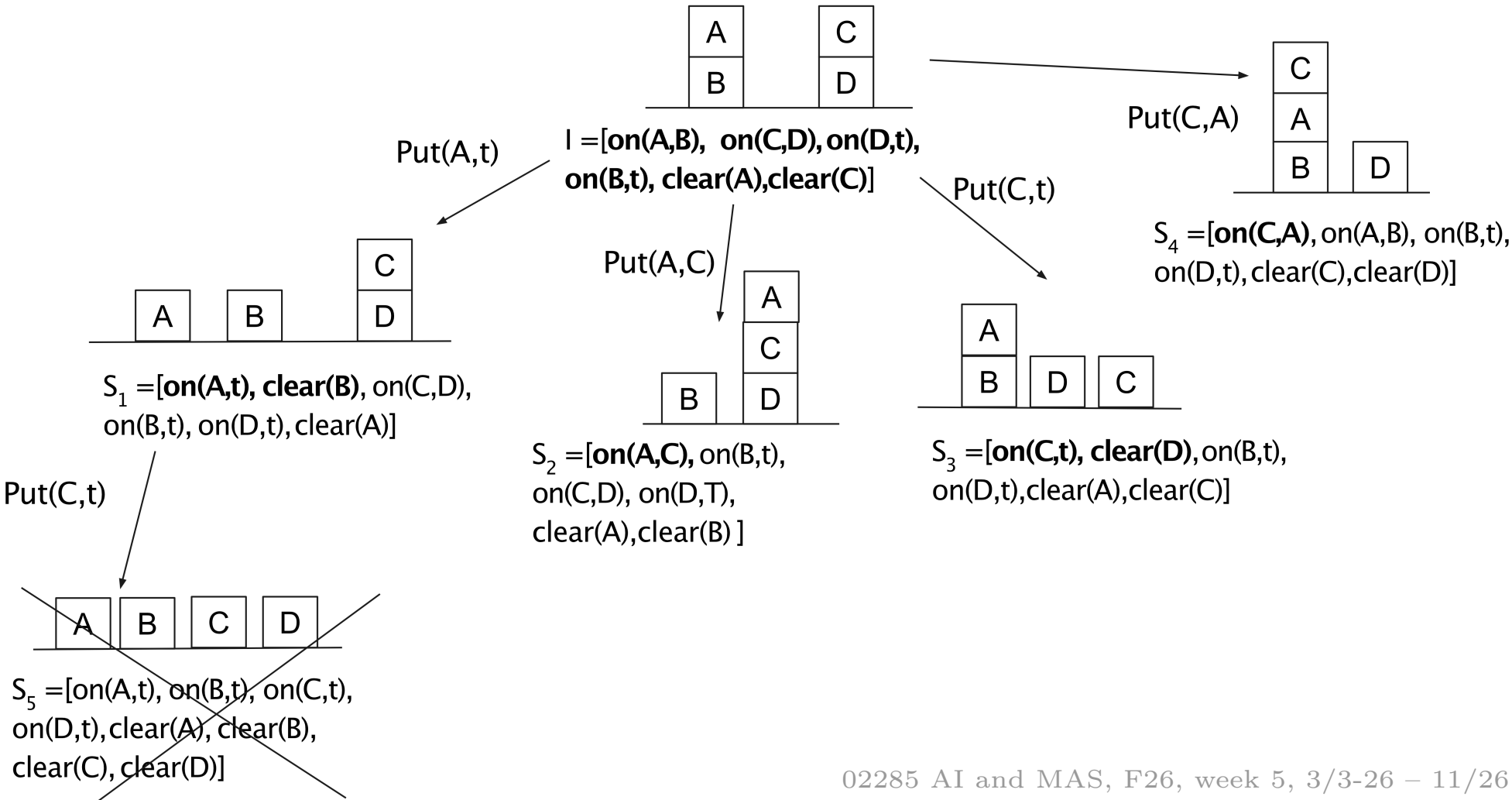
Example cont'd

We managed to solve the blocks world problem on the previous slide at width 1 (by running $IW(1)$). There are 3 blocks in the blocks world problem, so $|At| = 20$, cf. the earlier example about the number of atoms in blocks world problems.

Note that $IW(1)$ does a breadth-first search in a state space of size $|At| + 1 = 21$ instead of a standard BFS searching in a state space of size $2^{|At|} = 2^{20} \approx 10^6$. Linear instead of exponential in the number of atoms!

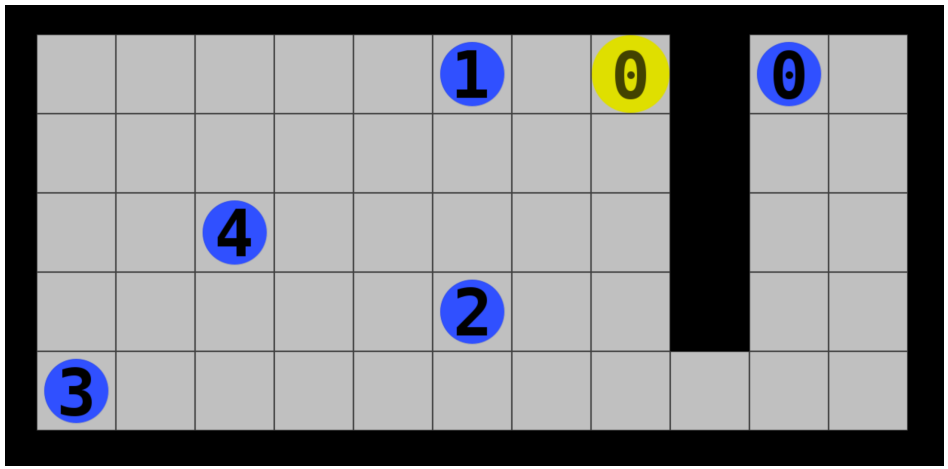
Another example (also by Frederik Drachmann)

Goal: $On(B, D)$. $IW(1)$ gets blocked: state s_5 is not added, and all other states would require to move A down, which we can't (since $On(A, Table)$ and all $Clear(x)$ already exist). In $IW(2)$, s_5 *is* added since it contains the new T^2 state $\{Clear(B), Clear(D)\}$: we find a solution.

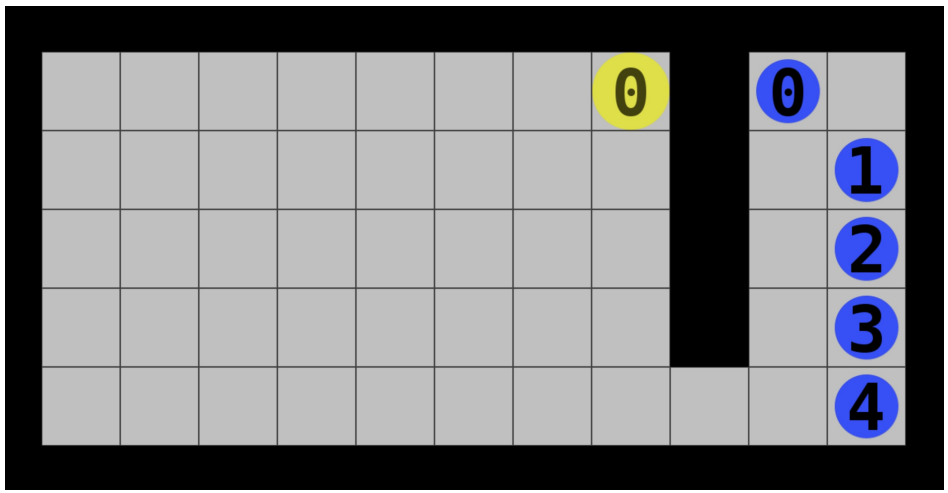


Testing IW on initial example

At which width do you think the following level will be solved? Width 1.



And what about the following level? Width 1 or at most 2 (see exercises for this week).



Non-optimality of IW

Consider the planning problem with initial state $s_0 = \{P(0), Q(0)\}$, goal $g = P(100)$ and action schemas:

ACTION *IncreaseP*(x)
PRECONDITION $P(x)$
EFFECT $P(x+1) \wedge \neg P(x)$

ACTION *IncreaseQ*(x)
PRECONDITION $Q(x)$
EFFECT $Q(x+1) \wedge \neg Q(x)$

ACTION *ShortCut*
PRECONDITION $P(1) \wedge Q(1)$
EFFECT $P(100)$

The optimal solutions are *IncreaseP*(1), *IncreaseQ*(1), *ShortCut* and *IncreaseQ*(1), *IncreaseP*(1), *ShortCut*. However, IW will find a longer solution than this. Which one and how long? In *IW*(1), we will have seen both $P(1)$ and $Q(1)$ at depth 1 in the tree, so we can't add the state $\{P(1), Q(1)\}$ at depth 2. So we can't exploit the *ShortCut* action and instead have to apply 100 *IncreaseP* actions.

Conclusion: IW is **not** an optimal algorithm.

Effective width

Recall: Iterated Width runs $IW(i)$ iteratively for increasing values of i.

Definition (Effective width)

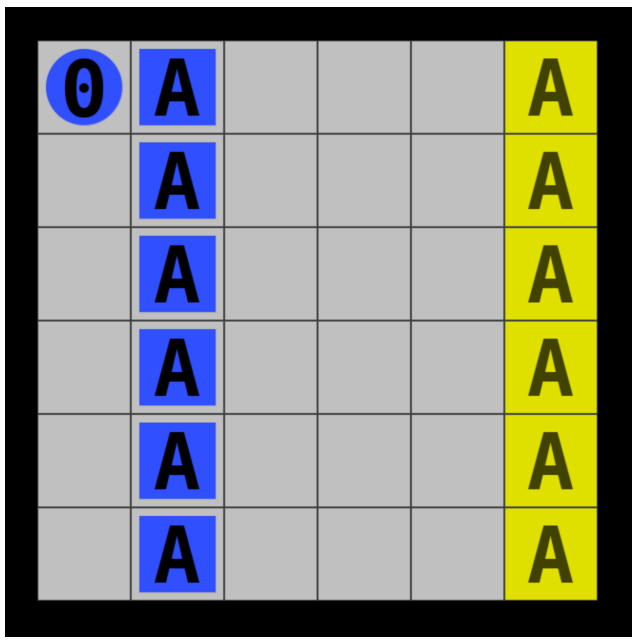
The first value i for which $IW(i)$ returns a solution is called the **effective width** of the problem.

In the planning problems from the International Planning Competition (IPC), there are 153 Sokoban levels. Approximately 1/3 of these have effective width 1, another 1/3 has effective width 2, and the rest has effective width > 2.

Iterated Width for conjunctive goals

Note that a planning problem with n subgoals is very likely to have effective width $\geq n$: Probably we can optimally achieve all combinations of $< n$ subgoals in fewer steps than achieving all n subgoals, and hence $IW(i)$ would block us from reaching the goal when $i < n$.

This would for instance most likely happen here:



Serialised Iterated Width (simple version)

A possible solution to the problem of conjunctive goals is Serialised Iterated Width (SIW):

Definition (Serialised Iterated Width, SIW (simple version))

We first run Iterated Width until reaching a state s where 1 subgoal has been achieved (any subgoal). We then rerun Iterated Width from s until reaching a state s' that achieves all previous subgoals as well as one additional subgoal. This continues until all subgoals have been reached. (Compare this with subgoal serialisability.)

The **effective width** of SIW on a problem is the max i for which a call to $IW(i)$ is made during the search.

Note: Simple SIW is not complete. Why not? The subgoals might not be serialisable in the order chosen by SIW. For instance, the first subgoal achieved might have to be undone to achieve the next.

We can try to be a bit smarter with the subgoal ordering...

Consistently achieving subgoals

Definition (Consistently achieving subgoals)

Let $P = (\mathcal{A}, s_0, g)$ be a planning problem, where we treat g as a conjunction of literals. Let $g' \subset g$ (a set of subgoals). A state s **consistently achieves** g' if s satisfies (all literals in) g' and there is a path from s to a state satisfying g that does not undo any subgoal in g'.

Checking whether a state s consistently achieves a set of subgoals g' can be as hard as solving the original planning problem, but a necessary condition for consistent achievability is tractable:

Lemma

Let $P = (\mathcal{A}, s_0, g)$ be a planning problem and s a state satisfying $g' \subseteq g$. Let P' be the modified planning problem where we delete any action that undoes a subgoal in g'. If $h_{max}(s) = \infty$ for P', then s can't consistently achieve g'.

Proof? $h_{max}(s)$ is admissible, so if $h_{max}(s) = \infty$, then the actual cost is infinite, i.e., the problem P' isn't solvable.

Serialised Iterated Width (full version)

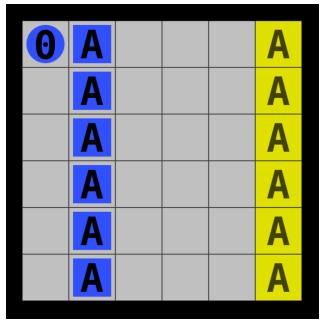
Definition (Serialised Iterated Width, SIW (full version))

We first run Iterated Width until reaching a state s where 1 subgoal has been consistently achieved (any subgoal). We then rerun Iterated Width from s until reaching a state s' that consistenctly achieves all previous subgoals as well as one additional subgoal. This continues until all subgoals have been reached.

The full version of SIW would have been complete if we could effectively compute consistent achievability. But we can't, and the original algorithm instead checks whether $h_{max}(s) < \infty$ for the modified problem P' where we delete any action that undoes a subgoal in g'. This is a necessary but not sufficient condition for consistent achievability, so SIW is not complete.

We can also consider a **strict version** of SIW, where no call to *IW* is allowed to undo a previously achieved subgoal. This makes it "more incomplete", but potentially more efficient (lower effective width).

Serialized Iterated Width example (strict SIW)

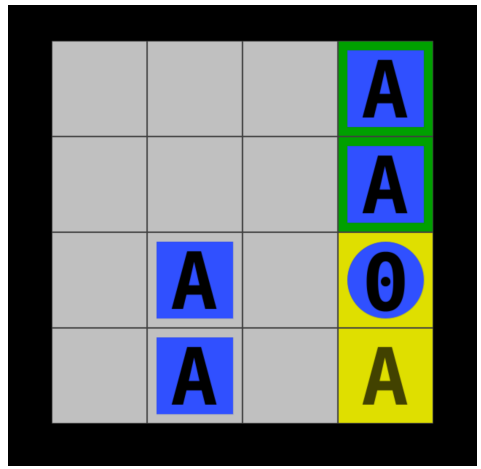


Consider strict SIW on this level. Similarly to the MAPF level, $IW(1)$ will find an optimal path to get the topmost box to its goal cell: no state on that optimal path will be blocked/pruned, and no other subgoal can be satisfied in fewer steps. In the resulting state, an optimal path to get the second box to its goal cell will not be blocked either in $IW(1)$ search. This continues until all boxes are on their goal cells.

We hence run $IW(1)$ n times where $n = \#\text{boxes}$. If we represent states by atoms $On(x, y, z)$ denoting that an object of type z is on location (x, y), then there are $6 \cdot 6 \cdot 2 = 72$ distinct atoms. We solve the level generating at most $72n$ states (compare to Warmup). Note that there is **no heuristic**, only a clever breadth-first search that has been serialised and work on substates of size 1.

More Serialized Iterated Width examples

Consider the following level reached by implementing SIW as a (relatively simple) modification of the client from the Warmup Assignment:



Can we solve the next goal with $IW(1)$? Yes. 0 is at $(3, 4)$ using the coordinate conventions from the Warmup client. 0 has to move to $(3, 3)$ to pick up the box. Then 0 cannot visit $(3, 3)$ again unless something else also changes. 0 does $Pull(N, E), Push(S, E)$ and is back in $(3, 3)$, but moved the box in both moves, so OK. Can we solve both remaining goals with one call of $IW(1)$? No. Solving the first requires 3 actions, and will hence move 0 into all cells reachable in 2 actions (we're doing BFS). When the first box is in its goal, 0 is in back in (3,3), but then all neighbouring cells have already been visited.

Efficiency of Iterated Width algorithms

Serialized Iterated Width is a very powerful algorithm. On the problems from the International Planning Competition, it solves even more problems, and solves them faster, than greedy best-first search with the h_{FF} heuristic, which is otherwise one of the most efficient domain-independent planning frameworks we have seen in this course.

Iterated Width can also be combined with greedy best-first search, to get the best of both worlds. This combination is called **Best-First Width Search** (Lipovetzky and Geffner, 2017). IW is here helping to get out of heuristic plateaus in greedy best-first search. At the most recent International Planning Competition, the two best performing planners were Fast Downward Stone Soup (mentioned earlier in the course) and Best-First Width Search.

BFS vs strict SIW on the hospital domain

level	BFS #generated	SIW #generated	effective width
SAD2	635,191	146	1
SAD3	10,101,051	143	1
MAPF01	2,351	94	1
MAPF02	110,446	138	1
MAPF03	5,063,874	184	1
SAsoko3_04	7,003	75	1
SAsoko3_05	974,806	161	1
SAsoko3_06	$> 50 \cdot 10^6$	307	1
SAFirefly	1,970,529	309	1
SACrunch	9,285,295	1993	2

The impressive efficiency of SIW comes partly from the **clever subgoal ordering** induced by the h_{max} check. Without it, `SACrunch` gives worst subgoal ordering (Why? A&D has lower cost, but block B&C.):

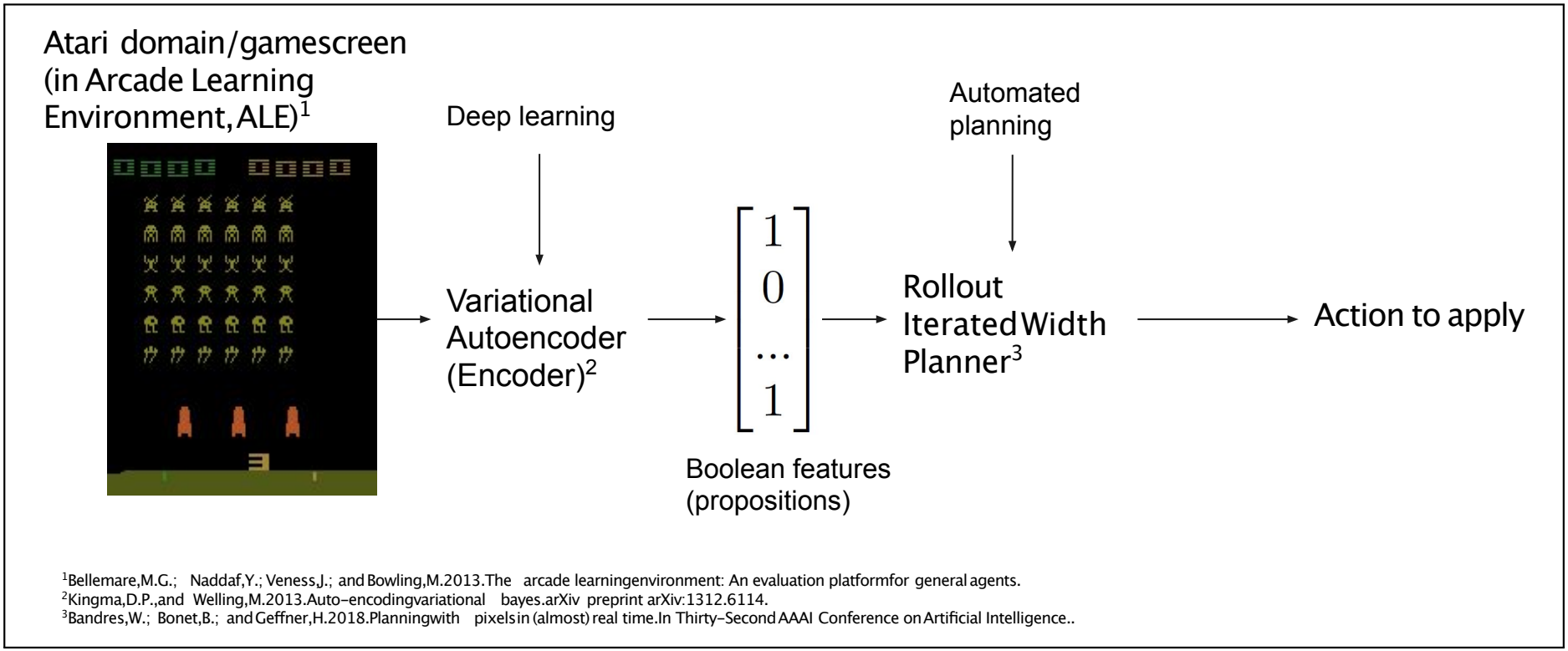
- No h_{max} check, no strict SIW: 571,166 states, eff. width 4 (way slower than BFS due to computational overhead per state).
- Include h_{max} check, but no strict SIW: 54,002 states, eff. width 3.

Adding a heuristic will improve things dramatically (not undo subgoals unnecessarily).

Machine learning + automated planning

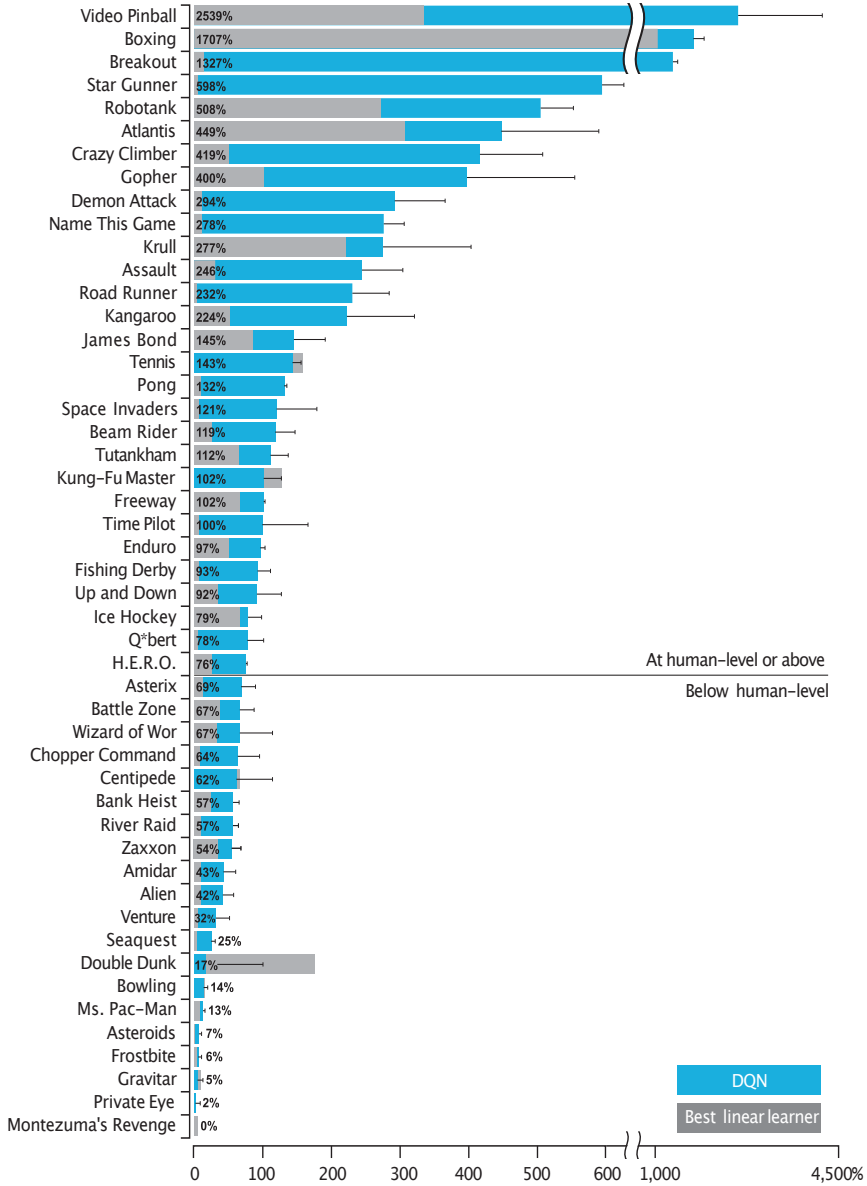
The recent years has seen a significant increase in systems that combine machine learning with automated planning, for instance using machine learning to learn action schemas, using machine learning to learn heuristics, etc.

Machine learning + Iterated Width

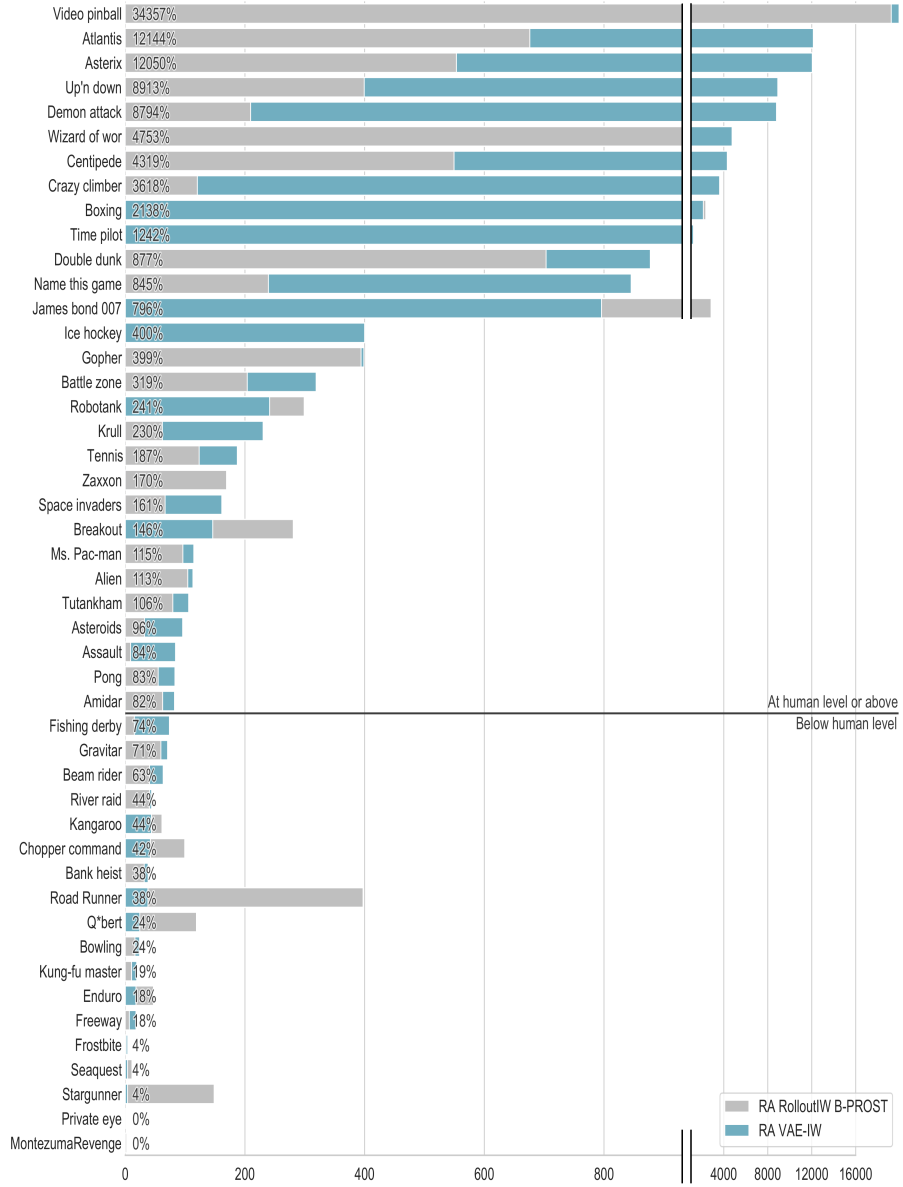


- Use variational autoencoders to reduce the screen pixel states of Atari video games to a set of (4500) boolean features, essentially turning pixels into (relatively) compact symbolic representations.
- Then plan on these representation using a roll-out version of Iterated Width (IW meets Monte-Carlo Tree Search).

(Frederik Drachmann, Andrea Dittadi, Thomas Bolander: Planning from Pixels in Atari with Learned Symbolic Representations, AAAI 2020)



(Mnih et al.: Human level control through deep reinforcement learning, Nature 2015)



(Drachmann, Dittadi, Bolander: Planning from Pixels in Atari with Learned Symbolic Representations, AAAI 2020)

Asterix results

Human playing Asterix

Our AI playing Asterix